Liberté • Égalité • Fraternité
RÉPUBLIQUE FRANÇAISE

MINISTÈRE CHARGÉ
DE L'EMPLOI

# DOSSIER PROFESSIONNEL (DP)


*Nom d'usage* ▸ SOINARD
*Prénom* ▸ Elia
*Adresse* ▸ 4 impasse des chênes 57370 PHALSBOURG


## Titre professionnel visé

**Concepteur Développeur d'Applications JavaScript Full-Stack**

## MODALITE D'ACCES :

☒ Parcours de formation
☐ Validation des Acquis de l'Expérience (VAE)

# Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen**.

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

## Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

*[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]*

## Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif)
- des annexes, si nécessaire.

*Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.*

**http://travail-emploi.gouv.fr/titres-professionnels**

# Sommaire

## Exemples de pratique professionnelle


**Développer une application sécurisée** **p.** 5

- Installer et configurer son environnement de travail en fonction du projet ..... p. 5
- Développer des interfaces utilisateur ..... p. 9
- Développer des composants métiers..... p. 13
- Contribuer à la gestion d'un projet informatique ..... p. 15

**Concevoir et développer une application sécurisée organisée en couches** **p.** 18

- Analyser les besoins et maquetter une application ..... p. 18
- Définir l'architecture logicielle d'une application ..... p. 22
- Concevoir et mettre en place une base de données relationnelle ..... p. 24
- Développer des composants d'accès aux données SQL et NoSQL ..... p. 27

**Préparer le déploiement d'une application sécurisée** **p.** 31

- Préparer et exécuter les plans de tests d'une application..... p. 31
- Préparer et documenter le déploiement d'une application ..... p. 36
- Contribuer à la mise en production dans une démarche DevOps ..... p. 42

**Titres, diplômes, CQP, attestations de formation** *(facultatif)* p. 46

**Déclaration sur l'honneur** p. 48

**Documents illustrant la pratique professionnelle** *(facultatif)* p.

**Annexes** *(Si le RC le prévoit)* p. 49

# EXEMPLES DE PRATIQUE PROFESSIONNELLE

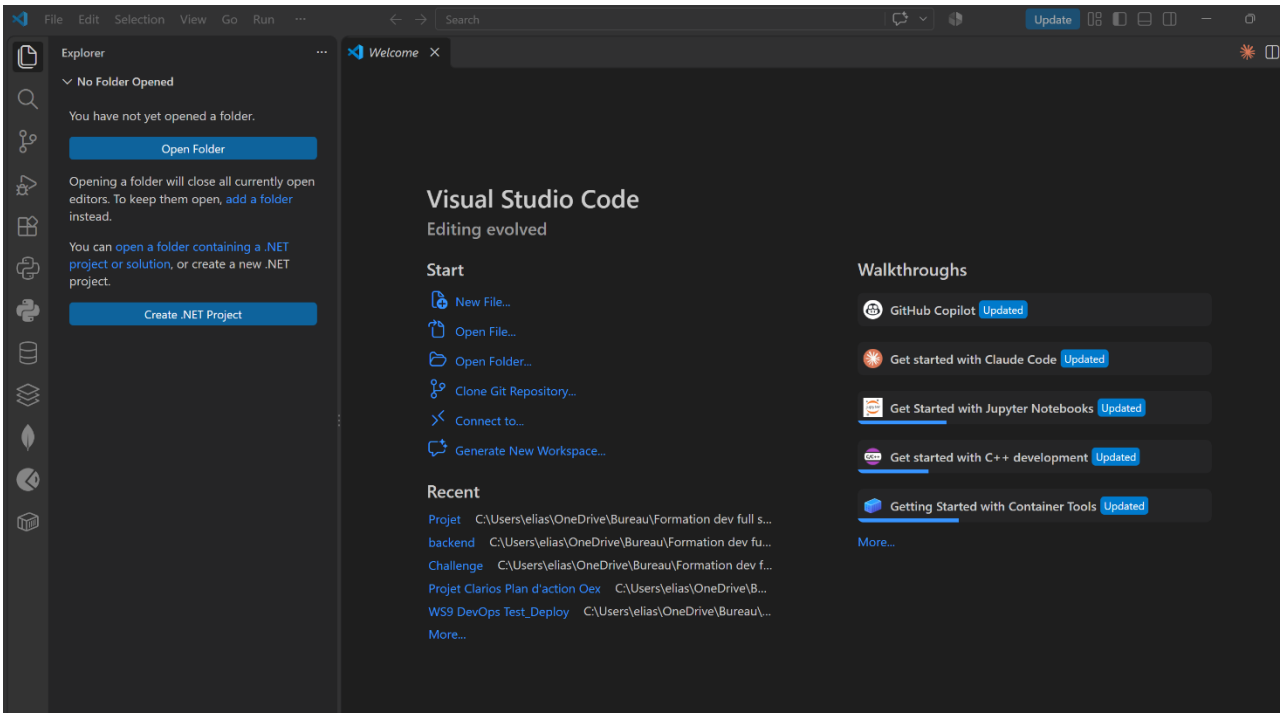
# Activité-type 1 Développer une application sécurisée

*Exemple n°1* ▸ *Installer et configurer son environnement de travail en fonction du projet*

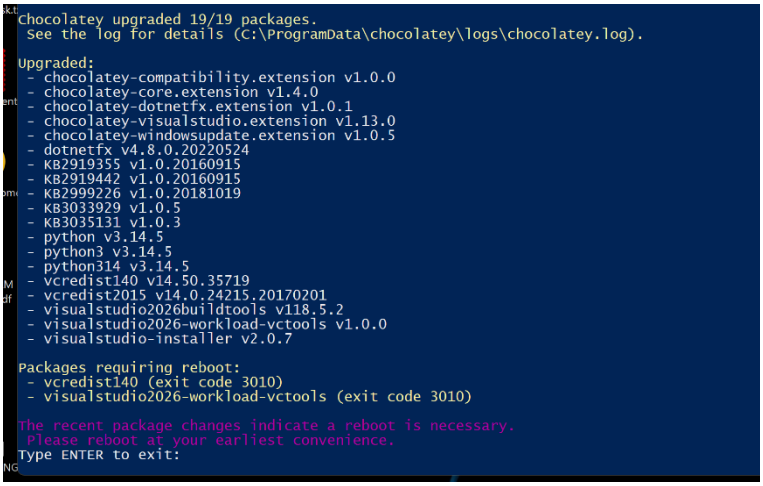
## 1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Pour le bon déroulement de mes différents projets, j'ai mis en place mon environnement de travail.

J'utilise Visual Studio Code comme éditeur principal




J'ai installé Node.js pour avoir accès à l'environnement JS côté serveur ainsi que le gestionnaire de paquets npm




J'ai installé gitBash comme terminal de commande.

```
Problems  Output  Debug Console  Terminal  Ports  Postman Console

elias@TABLET-SMIS623M MINGW64 ~/OneDrive/Bureau/Formation dev full stack/WS4 Création d'interfaces dyna
miques avec Javascript/Projet (master)
$ cd totolist

elias@TABLET-SMIS623M MINGW64 ~/OneDrive/Bureau/Formation dev full stack/WS4 Création d'interfaces dyna
miques avec Javascript/Projet/totolist (main)
$ npm start

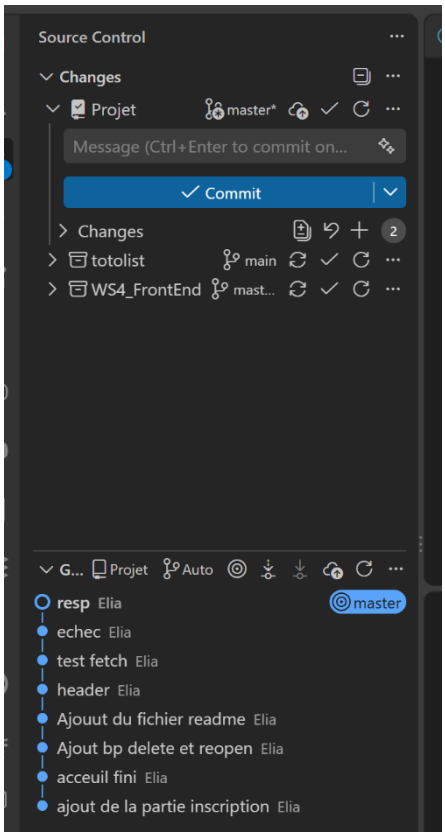
> qcm@1.0.0 start
> node app.js

Server is running on port 3000

*  History restored
```

J'ai installé Git et configuré ma connexion avec GitHub pour le suivi de versions et le partage de code. J'ai créé des dépôts distants pour mes différents projets, ce qui m'a permis de sauvegarder régulièrement mon travail à travers VS Code.

Je fais des commits régulier et à chaque grosse modification ou avancé dans le projet et les commentaires comporte un maximum d'informations pour que moi-même et les autres puissions-nous repérer au mieux.



J'ai installé diverses extensions pour chacun de mes projets :

- MongoDb pour créer des bases de données NoSQL
- MariaDB pour créer des bases de données SQL
- ESLint
- Postman pour tester mes routes côté Backend
- Docker

J'ai utilisé différents outils :

- Vite pour créer un projet REACT rapidement
- Vitest pour tester
- Express pour créer une API RestFull
- Swagger pour de la documentation
- Wave Evaluation Tool

**2. Précisez les moyens utilisés :**

J'ai utilisé une tablette surface pro7 et un petit écran additionnel qui fonctionne avec W11 avec 16Go Ram

**3. Avec qui avez-vous travaillé ?**

Je travaille en autonomie et j'ai été accompagné par mon formateur qui m'a guidé dans l'avancement de chaque projet, à éclairci des points que je ne comprenais pas et valider mes choix.

## 4. Contexte

**Nom de l'entreprise, organisme ou association** ▸ ***Global Digital University***

| | |
|---|---|
| **Chantier, atelier, service** | ▸ **Installer et configurer son environnement de travail en fonction du projet** |
| **Période d'exercice** | ▸ **Du 27/01/2026 au 27/09/2026** |

## 5. Informations complémentaires *(facultatif)*

Cliquez ici pour taper du texte.

## Activité-type 1 Développer une application sécurisée

*Exemple n°2 ▸ Développer des interfaces utilisateur*

### 1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre d'un projet consistant à créer une application de gestion de projet avec React, j'ai réalisé l'interface de l'application TaskFlow avec React et Tailwind CSS dans lequel j'y est mis du responsive et le mode dark.

J'ai choisi de montrer directement l'ensemble des projets directement après s'être connecté et au-dessus afficher les informations du User.

Les projets, informations user, tâches et tag sont sous forme de carte avec une bordure de couleur et une ombre pour la détacher du fond.

Logique UI carte :

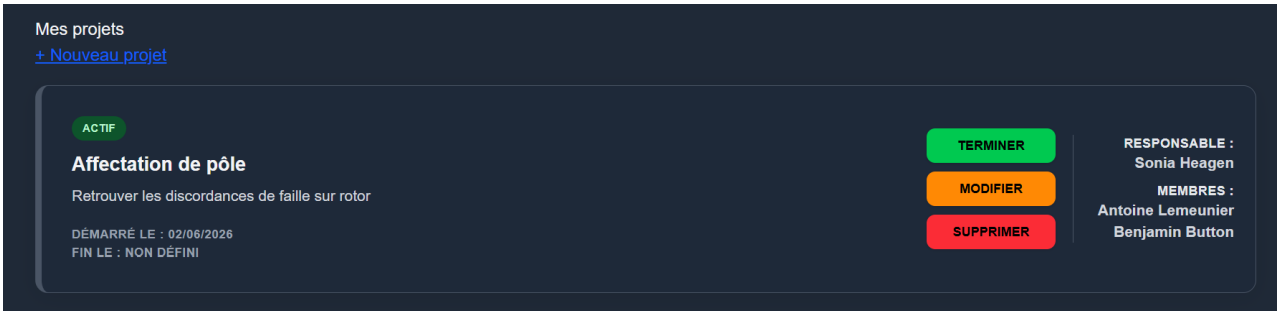
La carte se décompose en trois parties alignées horizontalement, la partie un est situé à gauche et comporte les informations techniques du projet, la partie deux est situé à côté est comporte les trois boutons est une barre verticale pour séparer les fonctionnalités et la partie trois comporte les informations des participants.

Logique UI partie une :
Tout en haut j'ai mis l'état du projet qui est vert pour actif et orange pour archiver, en-dessous en écriture grasse, de couleur blanche j'ai mis le titre du projet.
En-dessous du titre du projet j'ai mis le descriptif du projet en écriture normal, de couleur grise.
En-dessous j'ai aligné les deux informations comportant l'état de la priorité et l'échéance.
En dernier on retrouve les commentaires que j'ai mis dans une carte plus claire avec un border-radius pour adoucir les formes.

Logique UI partie deux :
Tous les boutons sont écrits en majuscule mais avec un code couleur pour chacun d'entre eux.
Vert pour le bouton « TERMINER » qui est une couleur de validation, orange pour le bouton « MODIFER » qui est une couleur d'avertissement et ROUGE » pour le bouton supprimer qui est une couleur qui signifie de devoir faire attention. J'y ai ajouter une barre verticale sur la droite pour séparer les boutons des informations.

Logique UI partie trois :

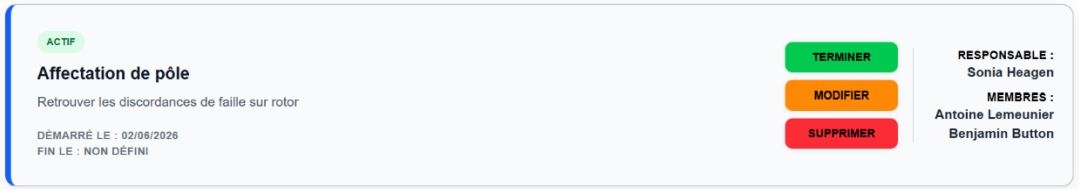
J'affiche tout à droite le responsable de projet et les membres associés.



Mode dark :
J'ai ajouté le mode dark pour le confort visuel des opérateurs.
Il est sélectionnable dans les paramètres du navigateur.
Avec le mode dark de sélectionné j'ai modifié le background du body et html et le background et color du main.

| | Body/HTML | | Main | |
|---|---|---|---|---|
| | Light | Dark | Light | Dark |
| **Background** | #f4f4f4 | #111827 | White | #1f2937 |
| **Color** | Non défini | | Non défini | #f3f4f6 |




Responsive :

Sur mobile, le menu se replie dans un bouton « hamburger ».
j'ai changé l'agencement de la carte pour que les trois parties gardent leurs contenant mais en les alignant verticalement ce qui me permet de rester fluide et de garder la même logique.
Photo avec un iPhone 16






React :
L'interface utilise des components qui sont utilisés dans les pages de navigation.
Au moment de la connexion je suis redirigé vers la vue des projets auquel j'utilise deux fonctionnalités de React.

- useEffect pour les requêtes API
- useState pour stocker les états

j'utilise la propriété « state » pour envoyer des informations dans les balises Link de React-router-dom et de les récupérer avec le hook useLocation de React-router-dom auquel j'utilise location.state pour mettre l'information dans une constante.

```jsx
import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import Task from "../components/Task";
import { get } from "../services/api";
import { useLocation } from 'react-router-dom';
//import Tags from "../pages/Tags";
import Tag from "../components/Tag";


function Tasks() {
    // 1. Déclaration unique des états
    const location = useLocation();
    // On extrait le projectId envoyé via le state du Link
    const projectId = location.state?.projectId;
    const projectTitle = location.state?.projectTitle;
    //const assigneeId = location.state?.assigneeId;
    //const assigneeName = location.state?.assigneeName;
```

```jsx
import { del, patch } from "../services/api";
import { Link } from "react-router-dom";

function Project({ project, onUpdate }) {

    const deleteProject = async () => {…
    };

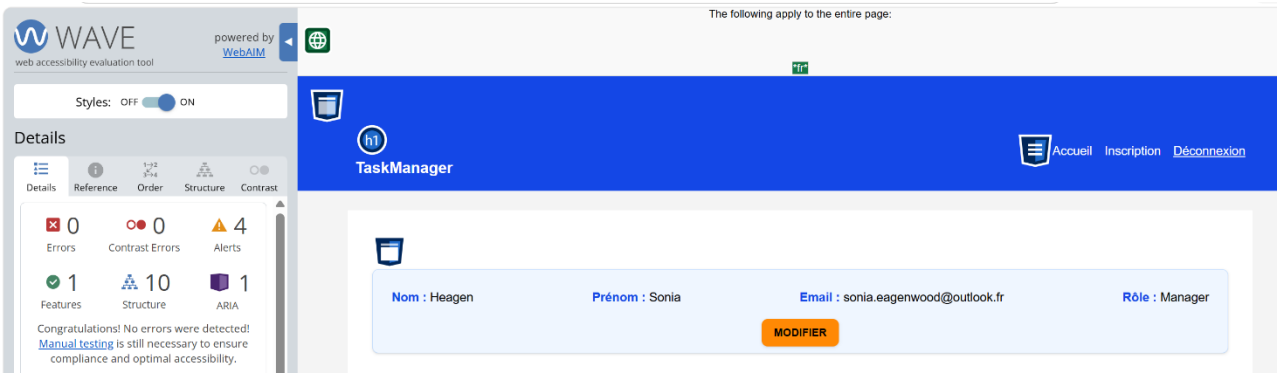
    const markAsFinished = async () => {…
    }

    return (
        <div key={project._id} className="bg-gray-50 dark:bg-gray-800 p-5 rounded-xl border border-gray-40

            {/* Colonne gauche */}
            <Link to="/tasks" state={{ projectId: project._id, projectTitle: project.title }} className="f
                <div className="flex items-center gap-2 mb-2">
```

Wave

J'ai utilisé le plugin wave pour évaluer mon rendu pour me permettre de l'améliorer.



## 2. Précisez les moyens utilisés :

React
Node.js
Bash
Tailwind
Wave

## 3. Avec qui avez-vous travaillé ?

**J'ai travaillé avec mon tuteur et l'IA Claude.**

## 4. Contexte

**Nom de l'entreprise, organisme ou association** ▸ ***Global Digital University***

| | |
|---|---|
| **Chantier, atelier, service** | ▸ **Développer des interfaces utilisateur** |
| **Période d'exercice** | ▸ **Du 07/07/2026 au 01/08/2026** |

## 5. Informations complémentaires *(facultatif)*

## Activité-type 1 Développer une application sécurisée

*Exemple n°3* ▸ *Développer des composants métier*

### 1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre d'un projet consistant à créer une application de gestion de projet avec Node.js et MongoDB j'ai créé des composants métier.
J'ai mis ces composants métiers dans le dossier « controllers ». Cette composante métier consiste à vérifier que l'utilisateur qui doit être un manager est autorisé pour supprimer un membre.
Dans le controllers je vérifier si le owner est bien le owner associé au projet.
S'il n'a pas les droits le statut 403 est retourné avec le message 'Suppression du membre non autorisé' et s'il a les droits je retire l'ID du membre du tableau members avec l'opérateur $pull.
Si la mise à jour fonctionne il y a le statut 200 avec le message 'Membre supprimé !' et si elle ne fonctionne pas il y aura une erreur de statut 400.
Si le serveur n'arrive pas à trouver de projet il renvoie null et l'erreur 500 sera retournée.

```
//Fonction Delete - Suppression d'un membre dans un projet
exports.deleteMembersProject = (req, res, next) => {
    Project.findOne({ _id: req.params.id })
        .then((project) => {
            // 1. Vérification des droits
            if (project.owner != req.auth.userId) {
                return res.status(403).json({ message: 'Suppression du membre non autorisé' });
            }

            // 2. Utilisation de $pull pour retirer l'ID du tableau 'members'
            Project.updateOne(
                { _id: req.params.id },
                { $pull: { members: req.body.members } }
            )
                .then(() => res.status(200).json({ message: 'Membre supprimé !' }))
                .catch(error => res.status(400).json({ error }));
        })
        .catch((error) => {
            res.status(500).json({ error });
        });
};
```

Le controllers est ensuite appelé dans une route sécurisée grâce à auth qui vérifie que mon token est valide et validateProject qui vérifie que l'ID des membres envoyés dans le body sont des ObjectId MongoDB valides ainsi que checkRole auquel j'ai spécifié 'ROLE_MANAGER' pour que seul un manager puisse supprimer un membre.

```
> /** ...
  //Supprimer un membre si les droits
  router.delete('/members/:id',auth, validate.validateProject, checkRole('ROLE_MANAGER'), multer, projectCtrl.deleteMembersProject);
```

**2. Précisez les moyens utilisés :**

Node .js
Express
Mongoose
JWT

**3. Avec qui avez-vous travaillé ?**

**J'ai travaillé avec mon tuteur et l'IA Claude.**

**4. Contexte**

**Nom de l'entreprise, organisme ou association** ▸ ***Global Digital University***

**Chantier, atelier, service** ▸ **Développer des composants métier**

**Période d'exercice** ▸ **Du 26/05/2026 au 22/06/2026**

**5. Informations complémentaires** *(facultatif)*

Cliquez ici pour taper du texte.

## Activité-type 1 Développer une application sécurisée

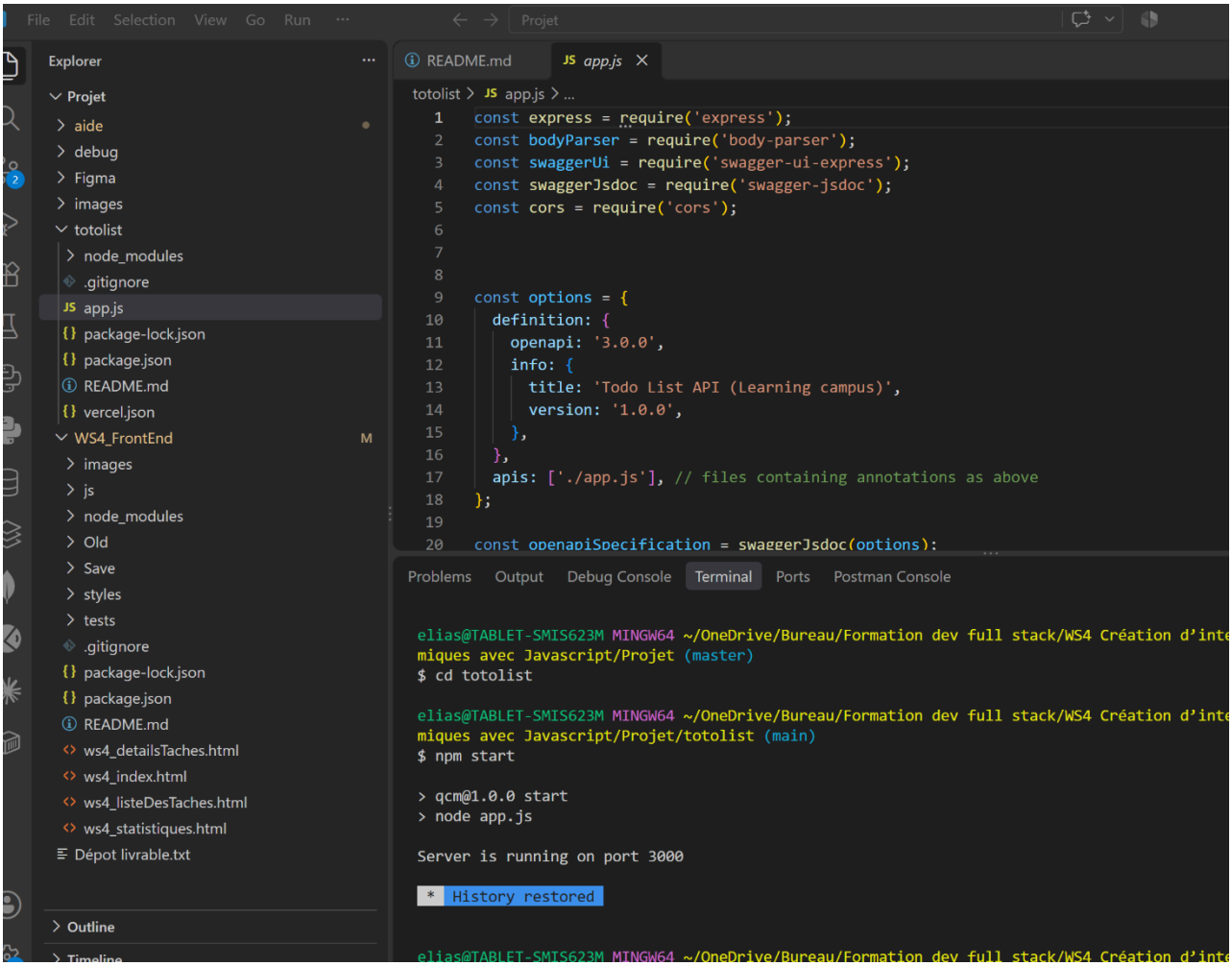
*Exemple n°4* ▸ *Contribuer à la gestion d'un projet informatique*

### 1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du projet WS4 Création d'interface dynamiques avec JavaScript, il y avait deux dépôts distincts : un dépôt Backend (API fournie, récupérée par git clone) et un dépôt Frontend que j'ai développé.

Devoir récupérer le dépôt du Backend m'a permis de le tester en local, analyser ma logique pour adapter mon Frontend.

J'ai pu m'organiser avec mon tuteur avec une réunion par semaine concernant l'avancement du projet, les points bloquants et les solutions que j'ai proposé




Cette séparation m'a permis de travailler sur l'interface sans modifier l'API, et de déployer chaque partie indépendamment : le front sur Vercel, le back sur Railway.

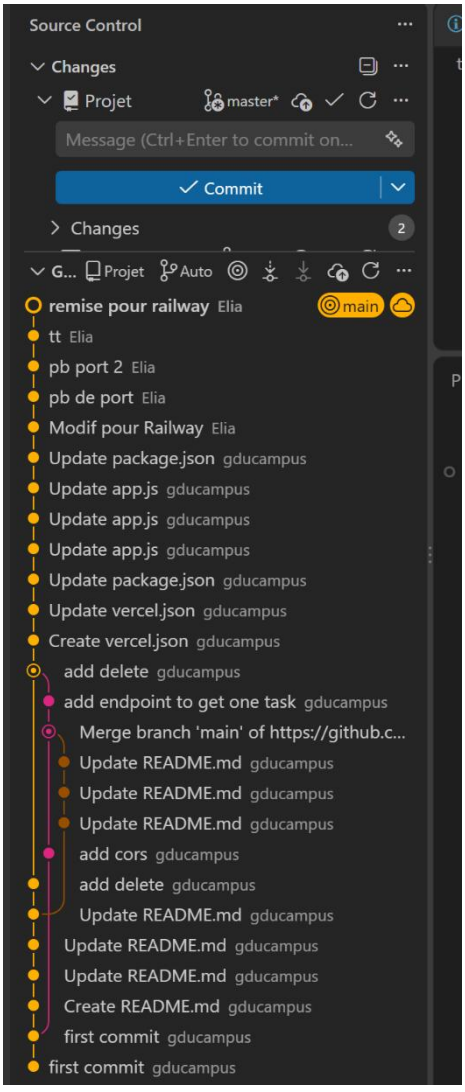
J'ai découpé le travail en plusieurs étapes :

1) J'ai lu les besoins client
2) J'ai récupéré le dépôt du Backend et je l'ai étudié
3) J'ai analysé le besoin client
4) J'ai fait la maquette Figma
5) J'ai créé mon Frontend en commençant par la page d'accueil puis la page des tâches puis le détail et j'ai fini avec la vue des statistiques
6) J'ai testé en local mon Frontend
7) J'ai déployé le Backend sur Railway et le Frontend sur Vercel

J'ai adapté le programme par rapport à la maquette car mon temps alloué c'est retrouvé rétréci et pour maintenir les délais j'ai simplifié la maquette.

J'ai mis sur le tableau de bord directement toutes les tâches sous forme de liste ainsi que le détail des tâches et l'ajout d'une tâche pour ne garder qu'une vue séparée pour les statistiques.

J'ai maintenu des commit régulier pour assurer le suivi du projet.



**2. Précisez les moyens utilisés :**

VS Code
Node.js
Railway
Vercel

**3. Avec qui avez-vous travaillé ?**

**J'ai travaillé avec mon tuteur pour l'organisation du projet qui m'a permis de cibler les points qui pouvait me prendre du temps.**

**4. Contexte**

**Nom de l'entreprise, organisme ou association** ▸ ***Global Digital University***

| | |
|---|---|
| **Chantier, atelier, service** | ▸ **Contribuer à la gestion d'un projet informatique** |
| **Période d'exercice** | ▸ **Du 02/03/2026 au 21/04/2026** |

**5. Informations complémentaires** *(facultatif)*

Cliquez ici pour taper du texte.

## Activité-type 2

**Concevoir et développer une application sécurisée organisée en couches**

***Exemple n°1*** ▸ *Analyser les besoins et maquetter une application*

### 1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du projet WS4 Création d'interface dynamiques avec JavaScript j'ai fait une maquette d'un projet Frontend pour me permettre de mieux analyser les besoins et me représenter le site futur.

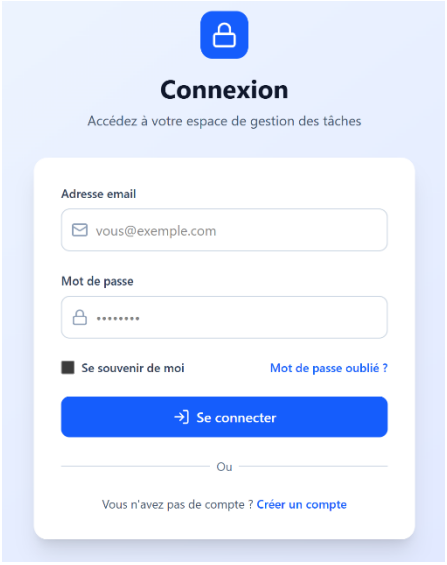
J'ai déterminé les besoins en analysant la demande client et j'ai remonté ces points :

- Une page d'accueil pour que l'utilisateur se connecte avec un bouton pour voir la liste des tâches et afficher la vue suivante
- Une page comportant les tâches de l'API Backend avec un bouton pour afficher les détails et ajouter une nouvelle tâche
- Une page comportant les détails d'une tâche auquel l'utilisateur peut la marqué comme terminé/rouvrir/supprimer
- Une page comportant les statistiques avec le nombre total de tâches, le nombre de tâche terminées et le restant de tâches à faire

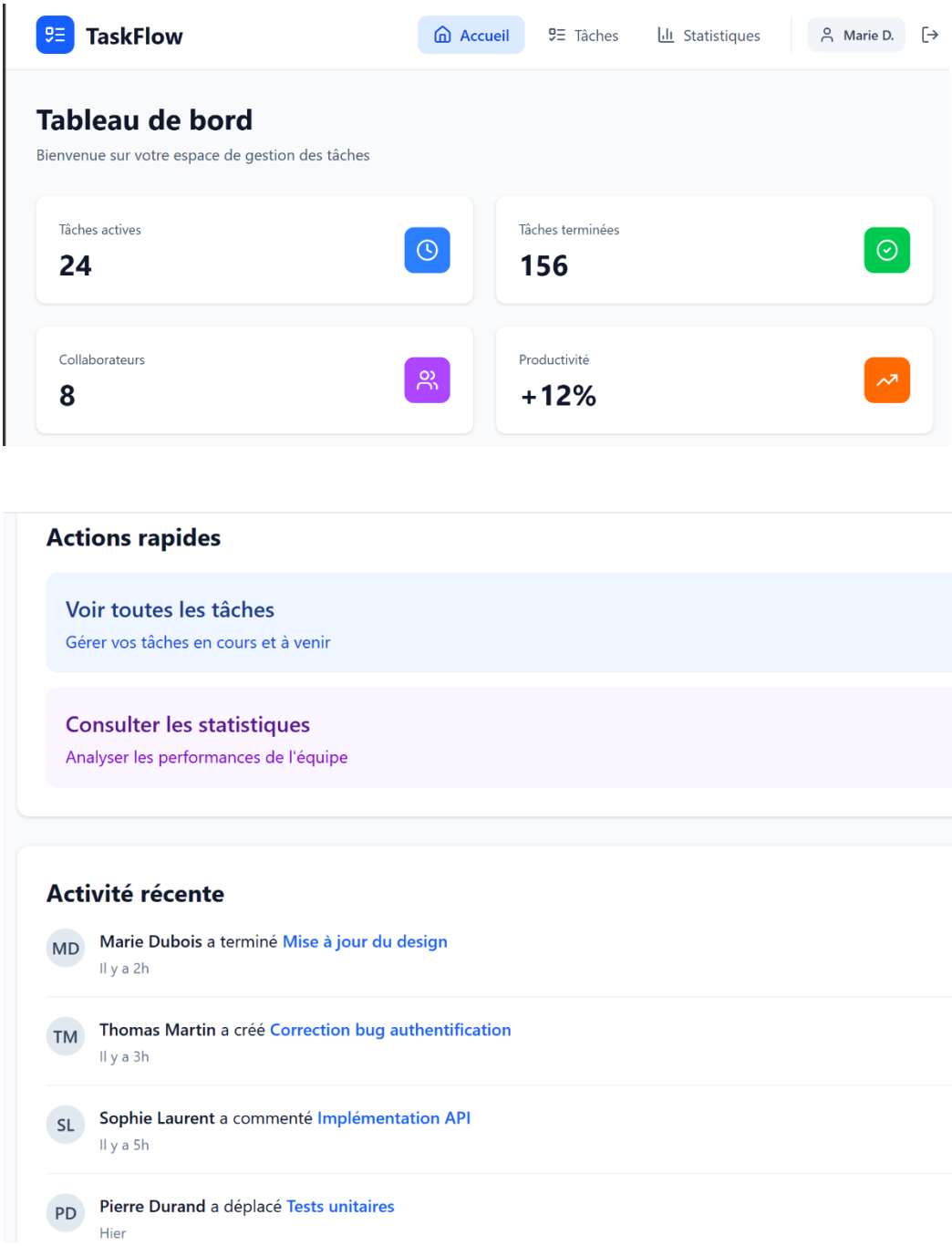
Une fois que j'ai fini d'analyser les besoins clients, j'ai fait une maquette sur figma. Cette maquette m'a servi à représenter au mieux le site et à me projeter dedans ce qui m'a aidé à organiser ma structure de développement logiciel.

Voici les différentes vues de la maquette Figma.

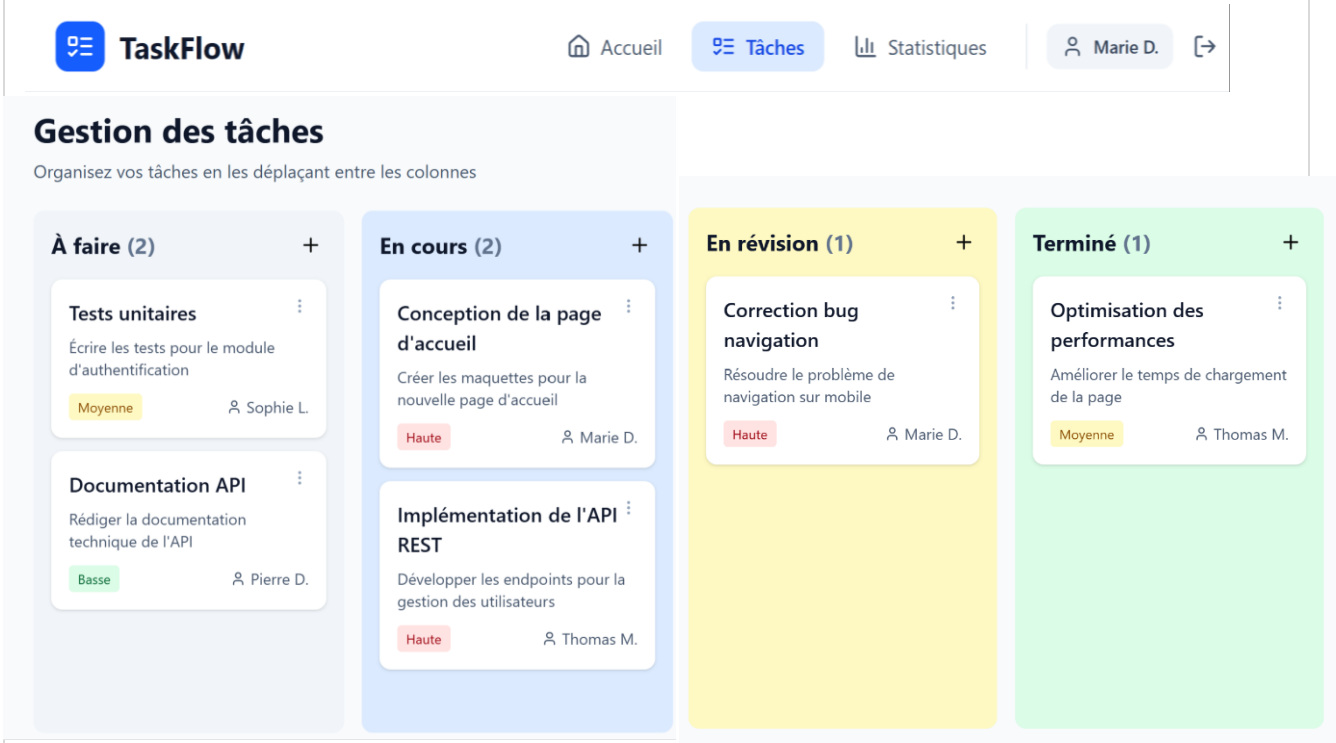
La vue d'accueil comportant l'adresse email et le mot de passe avec le bouton « Se connecter » qui renvoie l'utilisateur directement sur la vue des tâches.



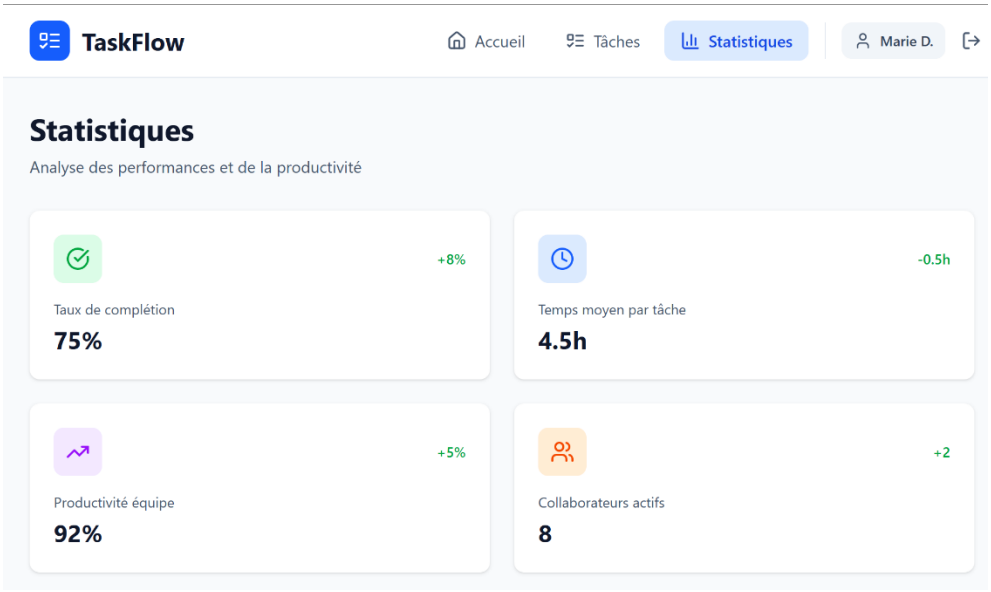

La vue comportant l'ensemble des tâches et affichant les premières infos comme les tâches actives/terminées, le nombre de collaborateur et le taux de productivité tout en permettant des actions rapides pour les tâches et statistiques.


TaskFlow
Accueil
Tâches
Statistiques
Marie D.
Tableau de bord
Bienvenue sur votre espace de gestion des tâches
Tâches actives
24
Tâches terminées
156
Collaborateurs
8
Productivité
+12%
Actions rapides
Voir toutes les tâches
Gérer vos tâches en cours et à venir
Consulter les statistiques
Analyser les performances de l'équipe
Activité récente
MD
Marie Dubois a terminé Mise à jour du design
Il y a 2h
TM
Thomas Martin a créé Correction bug authentification
Il y a 3h
SL
Sophie Laurent a commenté Implémentation API
Il y a 5h
PD
Pierre Durand a déplacé Tests unitaires
Hier

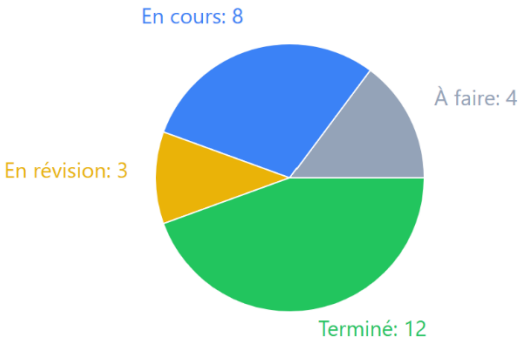
La vue comportant les détails dans lequel il est possible d'ajouter une tâche dans les quatre cas de figures (A faire, en cours, en révision, terminé)




La dernière vue comporte les statistiques ainsi qu'un camembert représentant l'état des tâches.



**Répartition des tâches par statut**




## 2. Précisez les moyens utilisés :

Le site Figma

## 3. Avec qui avez-vous travaillé ?

**J'ai travaillé avec mon tuteur et l'IA Claude.**

## 4. Contexte

**Nom de l'entreprise, organisme ou association** ▸ ***Global Digital University***

| | |
|---|---|
| **Chantier, atelier, service** | ▸ **Analyser les besoins et maquetter une application** |
| **Période d'exercice** | ▸ **Du 02/03/2026 au 21/04/2026** |

## 5. Informations complémentaires *(facultatif)*

Cliquez ici pour taper du texte.

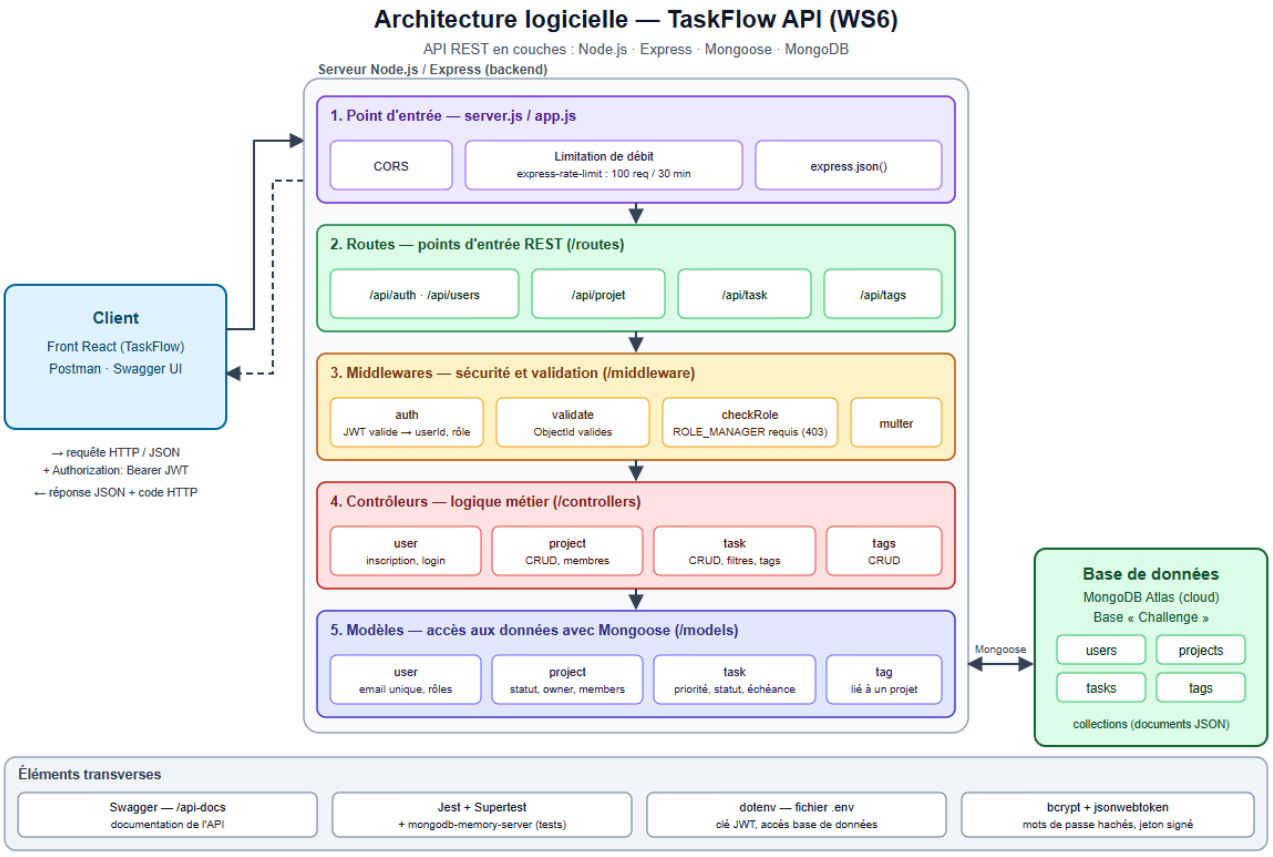
## Activité-type 2

**Concevoir et développer une application sécurisée organisée en couches**

***Exemple n°2*** ▸ *Définir l'architecture logicielle d'une application*

### 1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

L'API TaskFlow utilisé dans WS6 suit une architecture en couches. Chaque requête du client (front React, Postman ou Swagger UI) arrive au serveur Express, qui applique CORS et une limitation de débit. Elle est dirigée par les routes (/api/projet, /api/task, /api/tags, /api/users) vers les middlewares, qui vérifient le jeton JWT (auth), la validité des identifiants MongoDB (validate) et le rôle de l'utilisateur (checkRole). Le contrôleur applique ensuite les règles métier, par exemple le contrôle du propriétaire d'un projet. Il passe par les modèles Mongoose pour lire ou écrire dans MongoDB Atlas, puis la réponse repart en JSON avec le code HTTP adapté. Cette séparation en couches rend chaque partie plus simple à maintenir et à tester : les tests Jest et Supertest vérifient les accès et les rôles, et Swagger documente les routes.



**2. Précisez les moyens utilisés :**

Node.js
React
MongoDB

**3. Avec qui avez-vous travaillé ?**

**J'ai travaillé avec mon tuteur et l'IA Claude.**

**4. Contexte**

**Nom de l'entreprise, organisme ou association** ▸ ***Global Digital University***

**Chantier, atelier, service** ▸ **Définir l'architecture logicielle d'une application**

**Période d'exercice** ▸ **Du 26/05/2026 au 06/09/2026**

**5. Informations complémentaires** *(facultatif)*

Cliquez ici pour taper du texte.

## Activité-type 2

**Concevoir et développer une application sécurisée organisée en couches**

***Exemple n°3*** ▸ *Concevoir et mettre en place une base de données relationnelle*

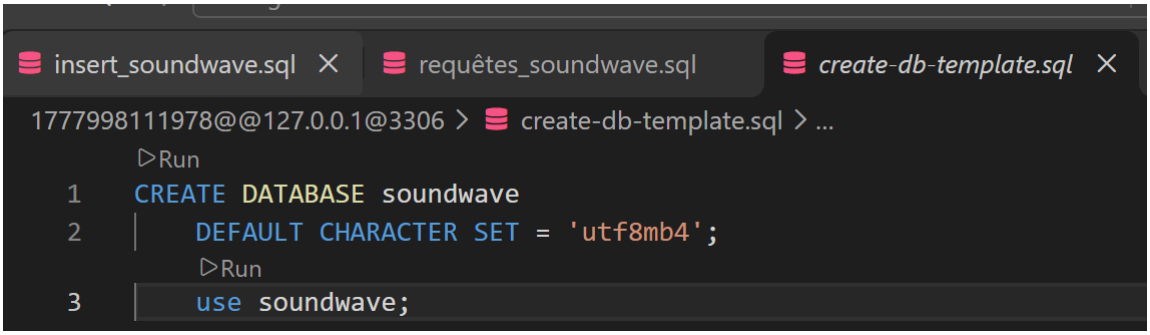
### 1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

**Données SQL**

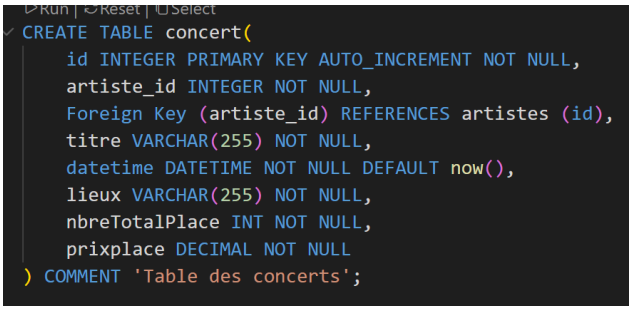
J'utilise l'IDE Visual Studio Code alors j'ai installé MariaDB sur la tablette et l'extension MySQL pour utiliser des tables de donnée SQL sur Visual Studio Code.




Ensuite j'ai créé ma base de données que j'ai nommé soundwave




J'ai créé plusieurs tables dans lequel j'ai spécifié les primary key et foreign key, le type de donnée, spécifié que je voulais qu'elles soient NOT NULL pour rendre les champs obligatoires et nommer chaque table.



```
Run | Reset | Select
CREATE TABLE concert(
    id INTEGER PRIMARY KEY AUTO_INCREMENT NOT NULL,
    artiste_id INTEGER NOT NULL,
    Foreign Key (artiste_id) REFERENCES artistes (id),
    titre VARCHAR(255) NOT NULL,
    datetime DATETIME NOT NULL DEFAULT now(),
    lieux VARCHAR(255) NOT NULL,
    nbreTotalPlace INT NOT NULL,
    prixplace DECIMAL NOT NULL
) COMMENT 'Table des concerts';
```

Liste de tables créer :

- clients
- artistes
- concert
- reservations

J'ai principalement utilisé les instructions JOIN, INSERT INTO, SELECT, DELETE, ALTER TABLE et j'ai utilisé les fonctions d'agrégations comme SUM et COUNT.

```
SELECT * FROM reservations
JOIN clients ON reservations.client_id = clients.id
JOIN concert ON reservations.concert_id = concert.id
WHERE clients.email = 'alenzojeremie@outlook.fr' LIMIT 100;
/* 2026-05-08 13:05:50 [10 ms] */
Run | Tab | JSON
SELECT concert.titre, SUM(reservations.qtplace) FROM reservations
JOIN concert ON reservations.concert_id = concert.id
GROUP BY concert.id LIMIT 100;
/* 2026-05-08 13:17:57 [25 ms] */
Run | Tab | JSON | Select
SELECT clients.nom, clients.prenom,COUNT(DISTINCT reservations.concert_id) AS nombre_concerts
FROM reservations
JOIN clients ON reservations.client_id = clients.id
GROUP BY clients.id
HAVING COUNT(DISTINCT reservations.concert_id) >= 2 LIMIT 100;
```

Exemple de la table des concerts

| id int(11) | artiste_id int(11) | titre varchar(255) | datetime datetime | lieux varchar(255) | nbreTotalPlace int(11) | prixplace decimal(10,0) |
|---|---|---|---|---|---|---|
| 9 | 2 | Vallée de la lune | 2026-05-08 10:54:13 | Senso-ji Temple | 512 | 12 |
| 10 | 1 | Snoozedate | 2042-01-12 00:12:13 | Mont Rushmore | 40000 | 23 |
| 11 | 3 | Paquerette du bison | 2014-05-04 08:30:00 | Teotihuacan, la cité des dieu | 18000 | 45 |
| 12 | 2 | Manipulation des ombres | 1989-10-08 02:10:05 | Conservatoire de musique c | 22000 | 130 |
| 13 | 1 | Dograve | 2026-05-08 16:42:55 | Centre spatial Kennedy | 430000 | 1200 |

## 2. Précisez les moyens utilisés :

Visual Studio Code
MariaDB

### 3. Avec qui avez-vous travaillé ?

**J'ai travaillé en autonomie et avec mon tuteur.**

### 4. Contexte

**Nom de l'entreprise, organisme ou association** ▸ ***Global Digital University***

**Chantier, atelier, service** ▸ **Concevoir et mettre en place une base de données relationnelle**

**Période d'exercice** ▸ **Du 08/04/2026 au 08/05/2026**

### 5. Informations complémentaires *(facultatif)*

## Activité-type 2

**Concevoir et développer une application sécurisée organisée en couches**

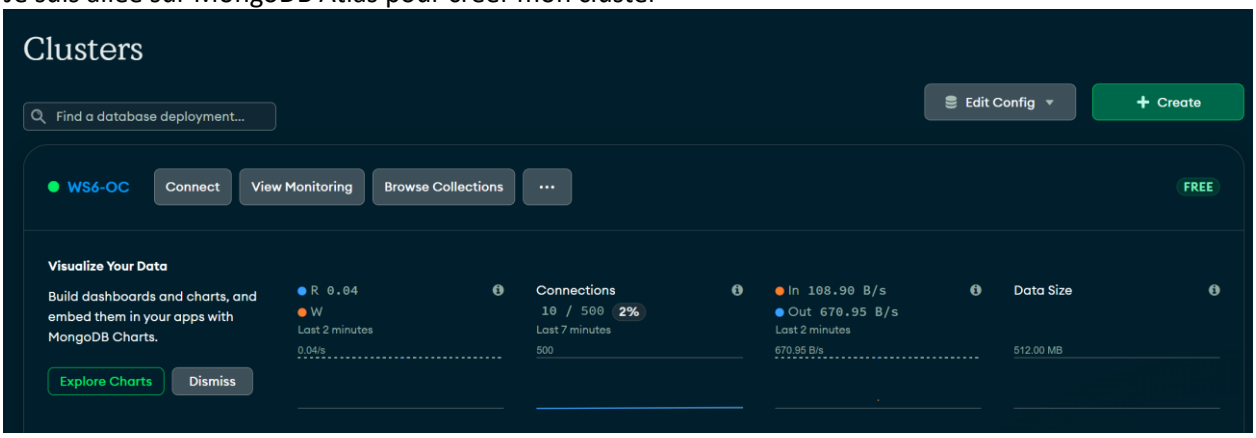
***Exemple n°4*** ▸ *Développer des composants d'accès aux données SQL et NoSQL*

### 1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'utilise l'IDE Visual Studio Code alors j'ai installé la bibliothèque Mongoose pour utiliser des collections sur Visual Studio Code via la commande npm et j'ai créé un compte sur le site de MongoDB Atlas.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE
elias@TABLET-SMIS623M MINGW64 ~/OneDrive/Bureau/Formation dev full stack/WS6 Créer une applicat
s et MongoDB/Challenge/backend
$ npm install mongoose
25 packages are looking for funding
  run `npm fund` for details
```

Je suis allée sur MongoDB Atlas pour créer mon cluster




Auquel j'ai fait le lien avec le backend en mettant le user et le password dans le fichier .env.

Le fichier .env étant appelé dans le gitignore il ne seront jamais envoyer sur GutHub.

```
backend > config > JS db.js > ...
1   const mongoose = require('mongoose');
2
3   const connectDB = async () => {
4       try { //Sa peut prendre du temps donc try/await
5           await mongoose.connect('mongodb+srv://'+process.env.DB_USER+':'+process.env.DB_PASSWORD+'@ws6-oc.k1ttgr3.mongodb.net/Challenge')
6               console.log('Connexion à MongoDB réussie !')
7       } catch (err) {
8           console.log('Connexion à MongoDB échouée !');
9           console.error(err);
10      };
11  };
12
13  module.exports = connectDB;
```

Dans mon programme j'ai créé un dossier models dans lequel j'ai mis les modèles de mes différentes collections avec la syntaxe Mongoose.

Exemple avec la collection project

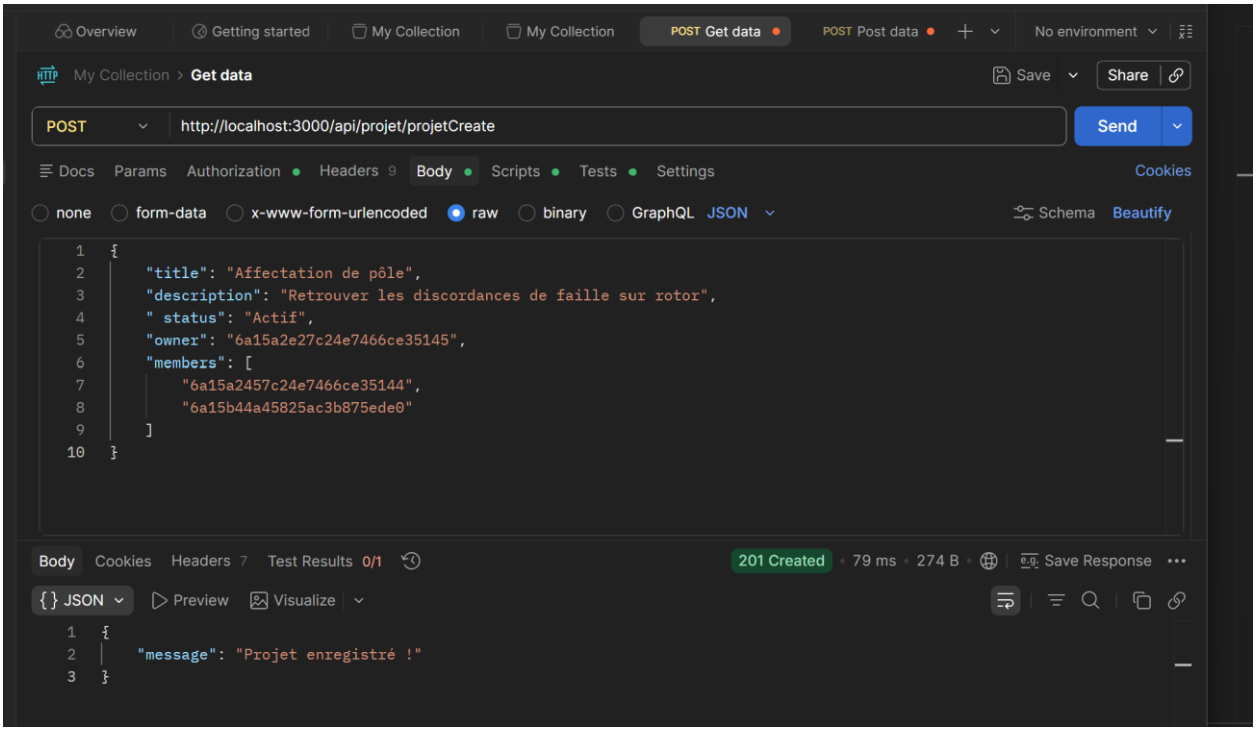
project.js

backend > models > project.js > ...

```
1   const mongoose = require('mongoose');
2   const user = require('./user');
3
4   const projectModel = new mongoose.Schema({
5       title: { type: String, required: true },
6       description: { type: String, required: true },
7       startAt:{type: Date, require:true, default: Date.now},
8       endAt:{type:Date, required:false},
9       status:{type: String}, //actif/archivé
10      owner: { type: mongoose.Schema.Types.ObjectId, ref: "user", required: true},
11      members: [{ type: mongoose.Schema.Types.ObjectId, ref: "user", required: true}]
12  });
13
14
15  module.exports = mongoose.model('project', projectModel); //Exporter le modèle avec les données
```

Puis j'ai utilisé le logiciel Postman pour tester mes routes Backend en utilisant le CRUD

Exemple pour une requête POST sur un projet



J'utilise le controller createProject pour créer un nouveau projet

```
//Fonction POST - Met tous/Création
exports.createProject = (req, res, next) => {
    const projectObject = req.body;
    const project = new Project({
        ...projectObject,
        userId: req.auth.userId
    });

    project.save()
        .then(() => { res.status(201).json({ message: 'Projet enregistré !' }) })
        .catch(error => { res.status(400).json({ error }) })
}
```

Qui sera créer avec la fonction POST du CRUD dans une route sécurisée grâce à auth qui vérifie que mon token est valide et validateProject qui vérifie que l'ID des membres envoyés dans le body sont des ObjectId MongoDB valides

```
//Met tous
router.post('/projetCreate', auth, validate.validateProject,multer, projectCtrl.createProject);
```

**2. Précisez les moyens utilisés :**

Visual Studio Code
MongoDB
Postman

## 3. Avec qui avez-vous travaillé ?

**J'ai travaillé avec mon tuteur et l'IA Claude**

## 4. Contexte

| | |
|---|---|
| **Nom de l'entreprise, organisme ou association** ▸ | ***Global Digital University*** |
| **Chantier, atelier, service** ▸ | **Développer des composants d'accès aux données SQL et NoSQL** |
| **Période d'exercice** ▸ | **Du 08/04/2026 au 08/05/2026** |

## 5. Informations complémentaires *(facultatif)*

## Activité-type 3 — Préparer le déploiement d'une application sécurisée

*Exemple n°1 ▸ Préparer et exécuter les plans de tests d'une application*

### 1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du projet WS9 DevOps Test & Déploiement, j'ai dû industrialiser l'application TaskFlow que j'avais développée en WS6 (API Node.js/Express/MongoDB) et en WS7 (interface React).
La première étape était de mettre en place des tests automatisés pour vérifier que l'application fonctionne et qu'elle reste sécurisée à chaque modification.

Préparation du plan de tests :
Avant d'écrire les tests, j'ai listé les exigences les plus importantes de l'application, en commençant par la sécurité :

- L'authentification : un utilisateur avec un token invalide ou sans token ne doit pas accéder aux routes protégées ;
- Les autorisations : seul un manager peut effectuer certaines actions ;
- La validation des données : une tâche sans titre, sans priorité ou avec un identifiant de projet invalide doit être refusée ;
- Les règles métier : on ne peut pas créer deux tags identiques ;
- Les fonctionnalités principales : création de projets et de tâches, formulaires, connexion.

Pour chaque exigence, j'ai défini le test à écrire et le résultat attendu.
Backend :

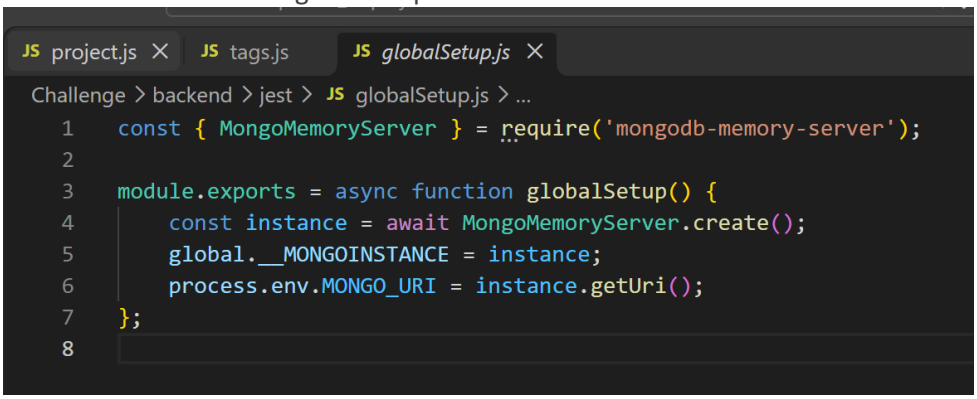
| Fichier | Test | Exigence couverte |
|---|---|---|
| jwtInvalide.test.js | retourne 401 si le JWT est invalide | Authentification |
| noAcces.test.js | doit refuser un utilisateur non connecté | Accès à une route protégée |
| project.test.js | création de projet | Création / consultation d'un projet |
| roleAutor.test.js | autorise un manager | Autorisations d'accès |
| roleNoAutor.test.js | refuse un utilisateur sans le rôle requis | Autorisations d'accès |
| task.test.js | Création de tâche | Création / modification d'une tâche |
| datavalidation.test.js | refuse une tâche sans titre (champ obligatoire) | Validation des données |
| datavalidation.test.js | refuse une tâche sans priorité (champ obligatoire) | Validation des données |
| datavalidation.test.js | refuse une tâche avec un ObjectId de projet invalide | Validation des données |
| tags.test.js | ne doit pas créer deux tags identiques | Règle métier (unicité) |

Frontend :

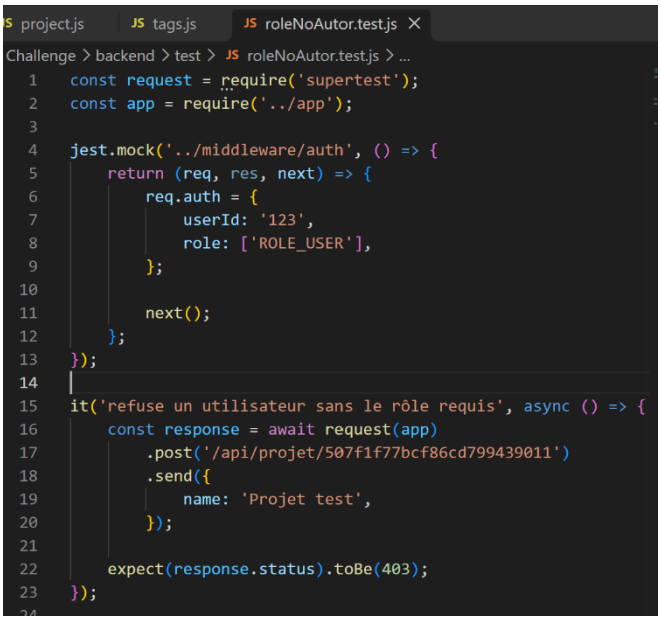
| Fichier | Test | Exigence couverte |
|---|---|---|
| login.test.jsx | Soumet le formulaire avec les bonnes valeurs | Composant React / Formulaire |
| login.test.jsx | Affiche une erreur si les identifiants sont incorrects | Affichage d'une erreur API |
| PrivateRoute.test.jsx | redirige vers /login si aucun token n'est présent | Route protégée |
| PrivateRoute.test.jsx | affiche le contenu protégé si un token est présent | Route protégée |
| CreateProject.test.jsx | Soumet le formulaire avec les bonnes valeurs | Fonctionnalité projet |
| EditProject.test.jsx | Soumet le formulaire avec les bonnes valeurs | Fonctionnalité projet |
| CreateTask.test.jsx | Soumet le formulaire avec les bonnes valeurs | Fonctionnalité tâche |
| EditTask.test.jsx | Soumet le formulaire avec les bonnes valeurs | Fonctionnalité tâche |
| CreateTag.test.jsx | Soumet le formulaire avec les bonnes valeurs | Composant React / Formulaire |
| EditTag.test.jsx | Soumet le formulaire avec les bonnes valeurs | Composant React / Formulaire |

J'ai ensuite choisi les outils selon la partie à tester :
Backend : Jest et Supertest. Supertest envoie de vraies requêtes HTTP à mon API et je vérifie le code de retour (401, 403, 200…).
Frontend : Vitest et React Testing Library. Je simule l'utilisateur qui remplit un formulaire et je vérifie le résultat affiché.

Pour que les tests ne touchent jamais la vraie base de données, j'ai utilisé mongodb-memory-server. À chaque lancement, une base MongoDB temporaire est créée en mémoire, puis supprimée à la fin.
Création de la base MongoDB temporaire avant les tests :



Exemple de test : refuser un utilisateur sans le rôle requis
Dans ce test, je simule un utilisateur connecté qui a seulement le rôle ROLE_USER. Il essaie d'ajouter un membre à un projet, une action réservée aux managers. Le résultat attendu est un refus avec le statut 403.




Côté frontend, j'ai testé ma route protégée PrivateRoute : sans token, l'utilisateur doit être redirigé vers /login, et avec un token, le contenu protégé doit s'afficher.
Exécution des tests :
J'ai lancé les tests avec npm test côté backend et npm run test côté frontend, tous les tests sont au vert :
Backend : 8 suites, 10 tests réussis ;
Frontend : 8 fichiers, 10 tests réussis ;
Au total, 20 tests réussis à 100 %.
Terminal backend avec les tests au vert

```
Test Suites: 8 passed, 8 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        25.947 s
Ran all test suites.
Force exiting Jest: Have you considered using `--detectOpenHandles` to detect async operations that kept running after all tests finished?
```

Terminal frontend avec les tests au vert

```
 ✓ src/__tests__/CreateTag.test.jsx (1 test) 95ms
 ✓ src/__tests__/login.test.jsx (2 tests) 111ms
 ✓ src/__tests__/EditTag.test.jsx (1 test) 100ms

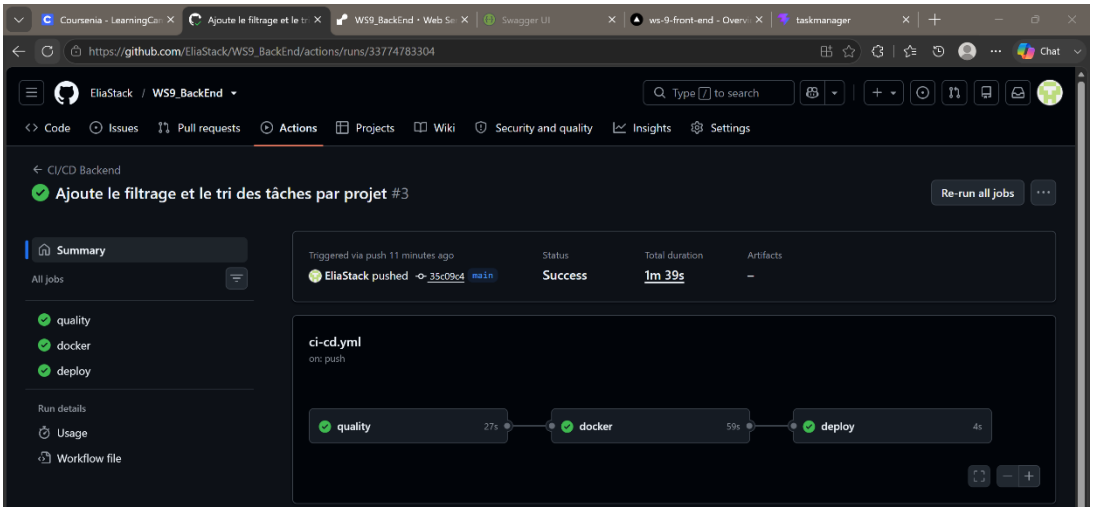
 Test Files  8 passed (8)
      Tests  10 passed (10)
   Start at  12:57:42
   Duration  31.84s (transform 960ms, setup 20.77s, import 1.45s, tests 882ms, environment 99.16s)

 PASS  Waiting for file changes...
       press h to show help, press q to quit
```

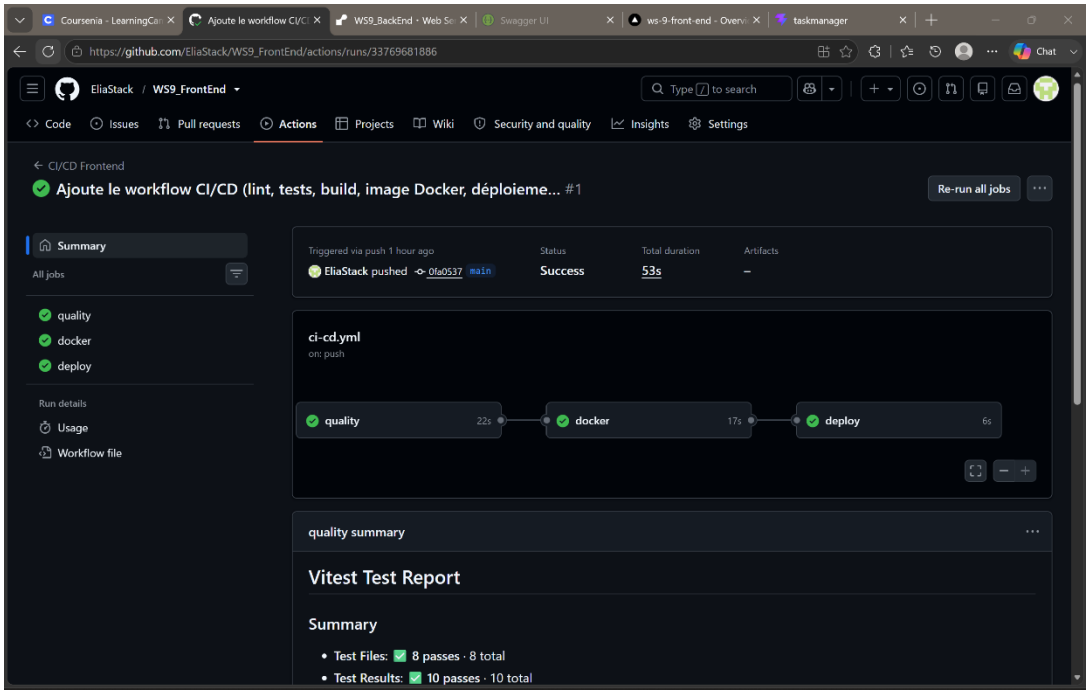
J'ai rédigé un rapport d'exécution des tests qui fait le lien entre chaque test et l'exigence qu'il couvre, et qui explique comment reproduire les résultats.

Pour ne pas avoir à lancer les tests à la main, je les ai intégrés dans un pipeline GitHub Actions (ci-cd.yml). À chaque push ou pull request sur la branche main, le pipeline installe le projet, vérifie le code avec ESLint, puis lance les tests. Si un test échoue, le déploiement ne se fait pas. Cela m'assure de ne jamais mettre en production une version qui casse une fonctionnalité existante.

Pipeline CI/CD backend : qualité (lint+tests), image Docker, déploiement.




Pipeline frontend avec le rapport Vitest.



**2. Précisez les moyens utilisés :**

**Visual Studio Code, Node.js, Jest, Supertest, mongodb-memory-server, Vitest, React Testing Library, ESLint, GitHub Actions**

**3. Avec qui avez-vous travaillé ?**

**J'ai travaillé avec mon tuteur et l'IA Claude**

**4. Contexte**

**Nom de l'entreprise, organisme ou association** ▸ ***Global Digital University***

**Chantier, atelier, service** ▸ **Préparer et exécuter les plans de tests d'une application**

**Période d'exercice** ▸ **Du 26/05/2026 au 06/09/2026**

**5. Informations complémentaires** *(facultatif)*

Cliquez ici pour taper du texte.

## Activité-type 3 — Préparer le déploiement d'une application sécurisée

*Exemple n°2* ▸ *Préparer et documenter le déploiement d'une application*

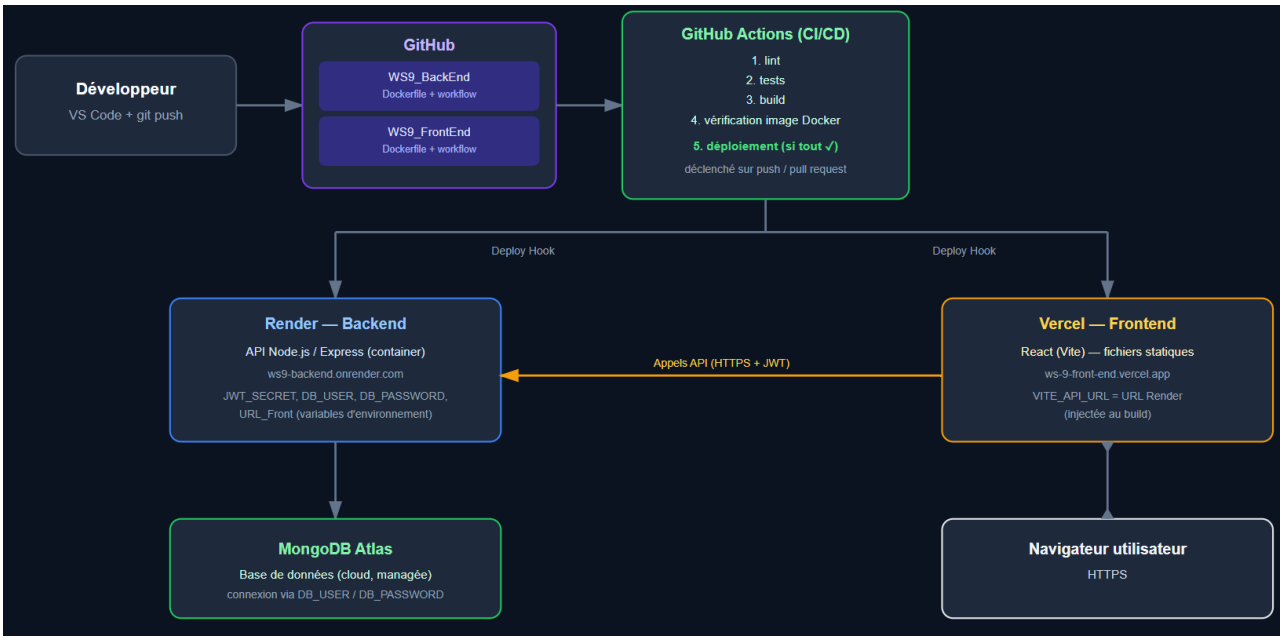
### 1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du projet WS9 DevOps Test & Déploiement, j'ai préparé la mise en ligne de l'application TaskFlow. Elle est composée de deux parties : une API Node.js/Express et une interface React. J'ai séparé le projet en deux dépôts GitHub distincts (WS9_BackEnd et WS9_FrontEnd) pour pouvoir déployer chaque partie indépendamment.

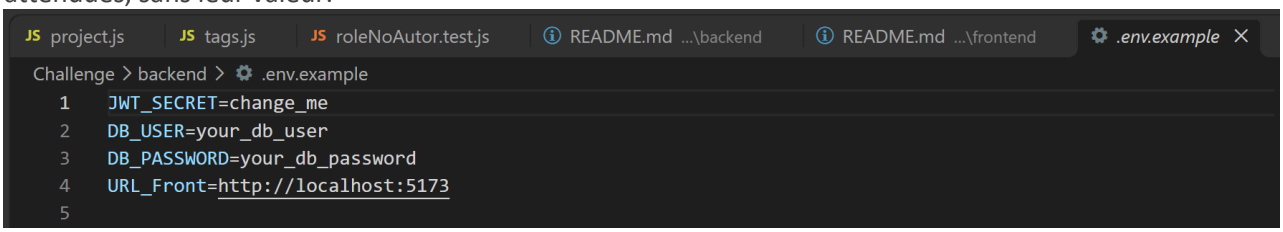
Choix de l'architecture de déploiement
J'ai choisi un hébergement adapté à chaque partie :

- Le frontend React sur Vercel, qui est spécialisé dans les sites statiques ;
- Le backend Node.js sur Render, qui fait tourner un serveur en continu ;
- La base de données sur MongoDB Atlas, une base managée dans le cloud.

J'ai représenté cette architecture dans un schéma. En développement, les trois mêmes composants tournent ensemble en local grâce à docker-compose.



Gestion des environnements et des variables :
J'ai séparé l'environnement de développement de l'environnement de production. Le backend charge un fichier différent selon la variable NODE_ENV : .env.dev en développement et .env.prod en production. Les données sensibles, comme la clé de signature des tokens JWT et les identifiants MongoDB Atlas, ne sont jamais écrites dans le code. Les fichiers .env sont exclus de Git grâce au .gitignore.
Pour que quelqu'un d'autre puisse installer le projet, j'ai créé un fichier .env.example qui liste les variables attendues, sans leur valeur.

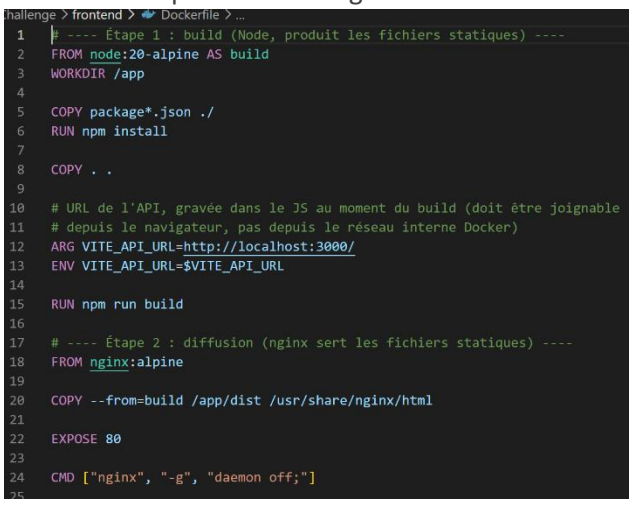



En production, j'ai renseigné ces variables directement dans les paramètres de Render et de Vercel.
Backend : variables d'environnement sur Render




Conteneurisation avec Docker :
Pour que l'application fonctionne de la même façon sur n'importe quelle machine, je l'ai conteneurisée avec Docker.

- Backend : un Dockerfile multi-étapes. L'étape de production installe uniquement les dépendances nécessaires (sans les outils de développement), et l'application tourne avec un utilisateur non-root pour limiter les risques en cas de faille.
- Frontend : un Dockerfile en deux étapes. Node.js construit l'application React (npm run build), puis un serveur nginx sert les fichiers produits. L'image finale est donc beaucoup plus légère.



J'ai ensuite écrit un fichier docker-compose.yml qui lance les trois services en une seule commande (docker compose up --build) : le backend, le frontend et une base MongoDB locale. La base n'est pas exposée à l'extérieur : seuls les autres conteneurs peuvent y accéder, par le réseau interne taskflow_network.

Fichier docker-compose.yml

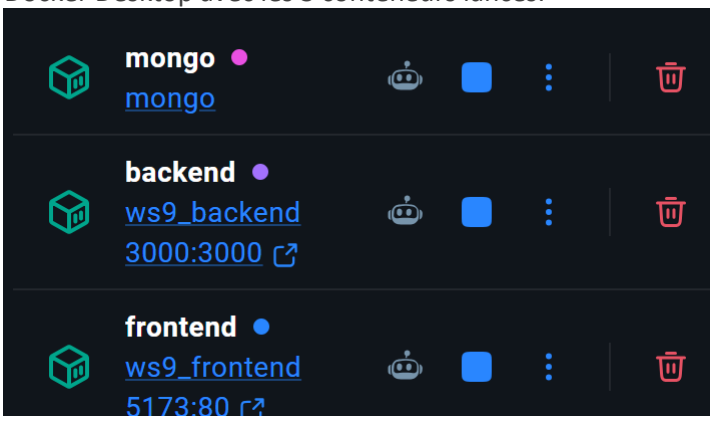
Challenge > docker-compose.yml

```yaml
Run All Services
services:
  Run Service
  mongo:
    image: mongo
    container_name: ws9_mongo
    volumes:
      - mongo_data:/data/db
    networks:
      - taskflow_network

  Run Service
  backend:
    build: ./backend
    image: ws9_backend
    container_name: ws9_backend
    environment:
      - MONGO_URI=mongodb://mongo:27017/Challenge
      - JWT_SECRET=dev_secret_docker
      - URL_Front=http://localhost:5173
    ports:
      - '3000:3000'
    depends_on:
      - mongo
    networks:
      - taskflow_network

  Run Service
  frontend:
    build:
      context: ./frontend
      args:
        VITE_API_URL: http://localhost:3000/
    image: ws9_frontend
    container_name: ws9_frontend
    ports:
      - '5173:80'
    depends_on:
      - backend
    networks:
```

Docker Desktop avec les 3 conteneurs lancés.



Qualité du code avant le déploiement :
Avant de déployer, j'ai mis en place ESLint pour détecter les erreurs dans le code et Prettier pour formater le code de manière uniforme (npm run lint et npm run format). J'ai corrigé les problèmes relevés et supprimé le code inutilisé.

Documentation du déploiement
J'ai rédigé un README qui documente tout le déploiement :

- L'architecture du projet et les URL de production ;
- L'installation en local, étape par étape ;
- Le tableau des variables d'environnement, avec leur rôle et leur portée (privée ou publique) ;
- La commande pour lancer les tests ;
- Le lancement avec Docker ;
- La procédure de déploiement ;
- La procédure de retour en arrière (rollback) en cas de problème : « Instant Rollback » sur Vercel, « Redeploy » d'une version précédente sur Render.

```
## Procédure de déploiement

Le déploiement est **automatisé** via GitHub Actions, séparément pour chaque dépôt.

**À chaque `push` ou `pull request` sur `main` :**

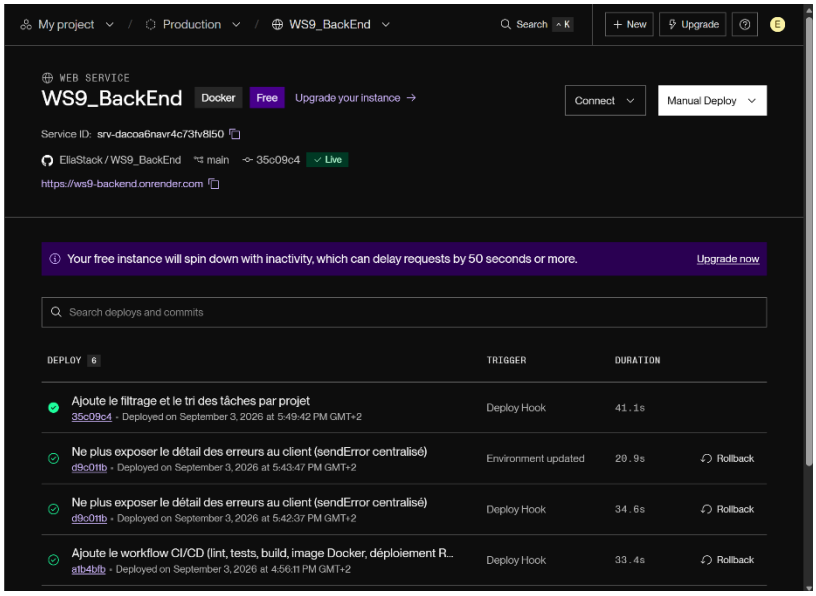
1. `npm ci` — installation des dépendances
2. `npm run lint` — contrôle qualité du code
3. `npm test` (backend) / `npm run test:run` (frontend) — exécution des tests
4. `npm run build` — vérification de la compilation
5. `docker build` — vérification que l'image Docker se construit correctement
6. **Si et seulement si toutes les étapes précédentes réussissent** : appel du Deploy Hook (Render pour
le backend, Vercel pour le frontend), qui déclenche le déploiement réel

Le pipeline échoue et **bloque le déploiement** si une seule de ces étapes échoue.

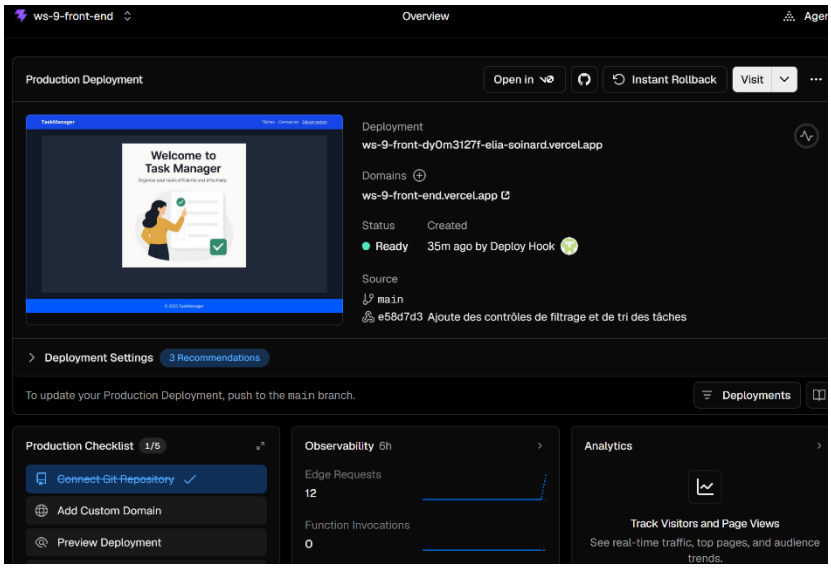
### Retour en arrière (rollback)

- **Vercel** : bouton "Instant Rollback" dans le dashboard du projet, sur n'importe quel déploiement
précédent marqué "Ready".
- **Render** : onglet "Deploys" du service → sélectionner un déploiement antérieur réussi → "Redeploy".
```

Mise en ligne :
L'application est en ligne : le frontend sur Vercel et l'API sur Render, avec la documentation Swagger accessible en ligne.
Backend :




Frontend :



## 2. Précisez les moyens utilisés :

**Visual Studio Code, Node.js, Docker, Docker Compose, nginx, ESLint, Prettier, GitHub, Render, Vercel, MongoDB Atlas**

## 3. Avec qui avez-vous travaillé ?

**J'ai travaillé avec mon tuteur et l'IA Claude**

## 4. Contexte

**Nom de l'entreprise, organisme ou association** ▸ ***Global Digital University***

**Chantier, atelier, service** ▸ **Préparer et documenter le déploiement d'une application**

**Période d'exercice** ▸ **Du 26/05/2026 au 06/09/2026**

## 5. Informations complémentaires *(facultatif)*

Cliquez ici pour taper du texte.

## Activité-type 3 — Préparer le déploiement d'une application sécurisée

*Exemple n°3 ▸ Contribuer à la mise en production dans une démarche DevOps*

### 1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du projet WS9 DevOps Test & Déploiement, j'ai automatisé la mise en production de l'application TaskFlow. L'objectif de la démarche DevOps est que chaque modification du code soit vérifiée puis déployée automatiquement, sans étape manuelle qui pourrait être oubliée ou mal faite.

Mise en place du pipeline CI/CD :

J'ai créé un workflow GitHub Actions (.github/workflows/ci-cd.yml) dans chacun de mes deux dépôts, WS9_BackEnd et WS9_FrontEnd. Il se déclenche à chaque push ou pull request sur la branche main.

Le pipeline est découpé en trois jobs qui s'enchaînent :

1. quality : installation des dépendances (npm ci), vérification du code avec ESLint, exécution des tests et vérification que le projet se construit sans erreur ;
2. docker : construction de l'image Docker, pour vérifier qu'elle fonctionne avant la mise en ligne ;
3. deploy : déclenchement du déploiement sur Render pour le backend, sur Vercel pour le frontend.

J'ai relié les jobs avec le mot-clé needs : le job docker ne démarre que si quality a réussi, et deploy seulement si les deux précédents ont réussi. Une seule erreur arrête donc toute la chaîne, et une version défectueuse ne peut pas arriver en production.

Le job deploy a aussi une condition : il ne s'exécute que lors d'un push sur main. Une pull request lance les vérifications, mais ne déploie rien.

Fichier ci-cd.yml du backend



```yaml
name: CI/CD Backend

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 20

      - run: npm ci

      - run: npm run lint

      - run: npm run build

      - run: npm test

  docker:
    needs: quality
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

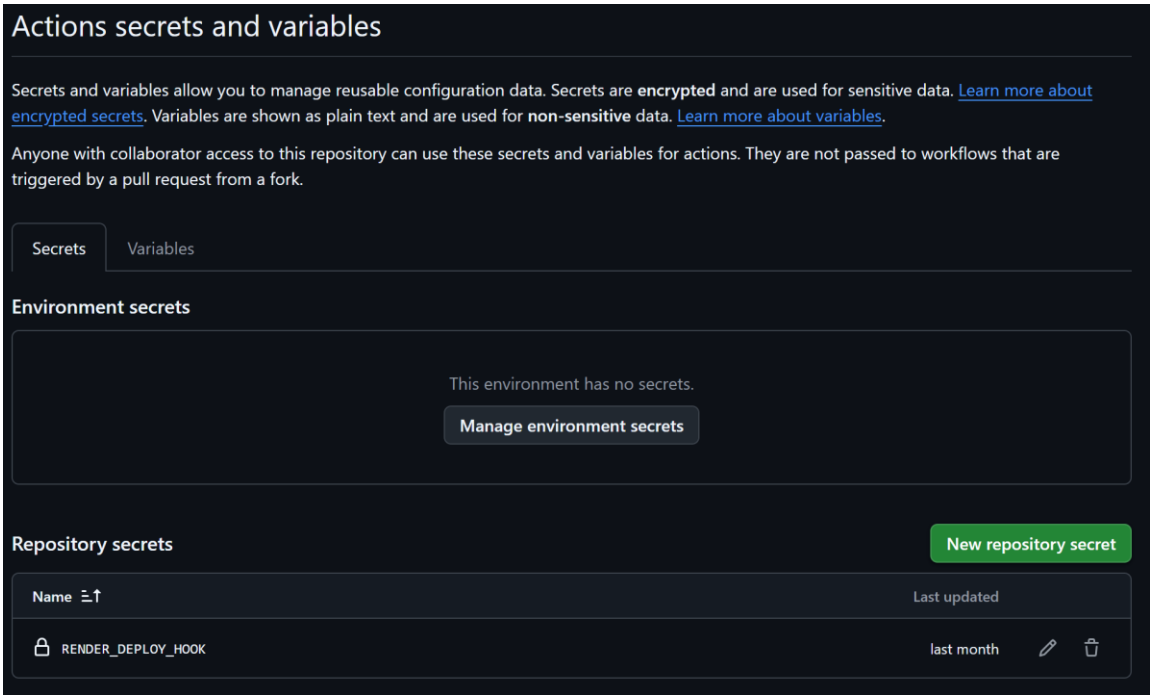
      - name: Construire l'image Docker (stage production)
        run: docker build --target production -t ws9_backend .

  deploy:
    needs: [quality, docker]
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - name: Déclencher le déploiement Render
        run: curl -fsS -X POST "${{ secrets.RENDER_DEPLOY_HOOK }}"
```

Sécurisation du déploiement :

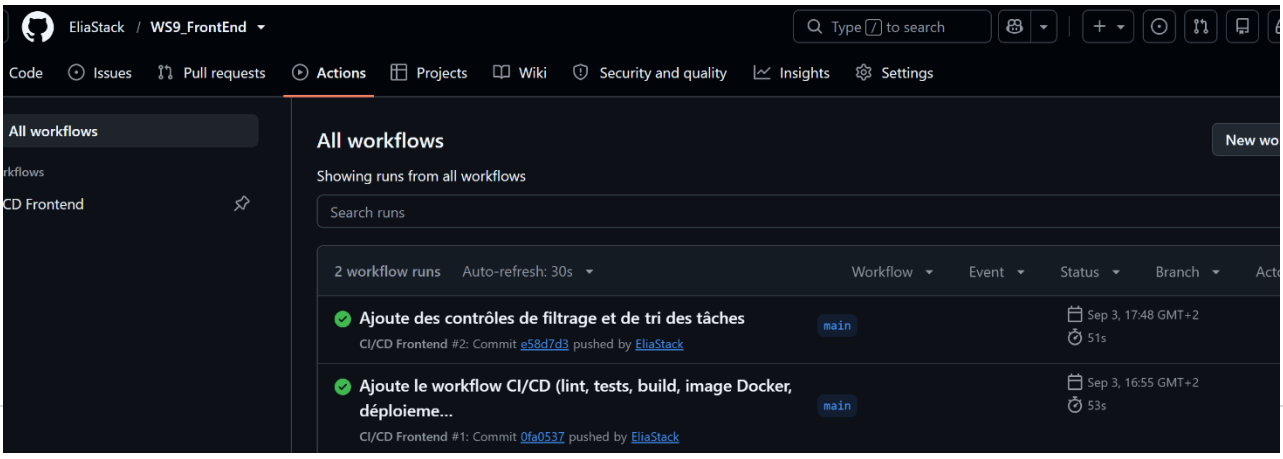
Pour déclencher le déploiement, le pipeline appelle un Deploy Hook : une adresse fournie par Render et par Vercel qui lance une nouvelle mise en ligne. Cette adresse donne le droit de déployer l'application, je ne l'ai donc pas écrite dans le code. Je l'ai enregistrée dans les secrets de GitHub (RENDER_DEPLOY_HOOK et VERCEL_DEPLOY_HOOK), et le workflow la lit avec ${{ secrets.... }}. Elle n'apparaît ni dans le dépôt ni dans les logs.

Vue Settings > Secrets and variables > Actions, avec le nom du secret

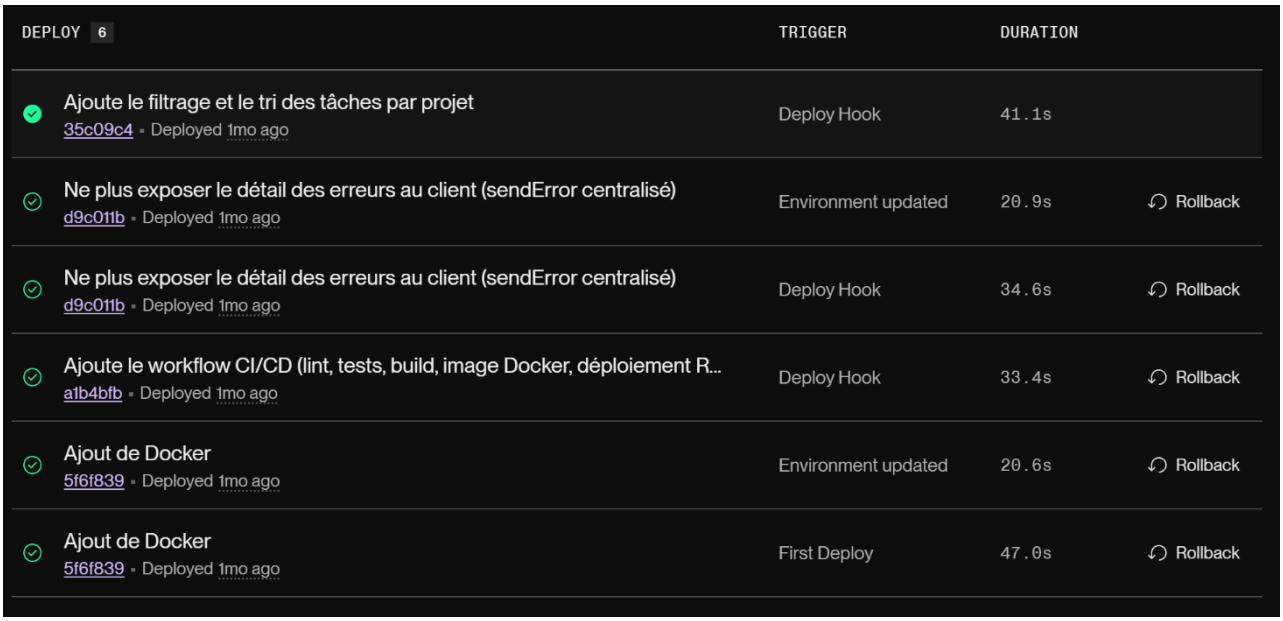



Un cycle complet, du code à la production
Pour la fonctionnalité de filtrage et de tri des tâches par projet, j'ai fait une modification sur le backend et une sur le frontend. Après chaque push sur main :
GitHub Actions a lancé le pipeline : lint, tests, build, image Docker ;
toutes les étapes étant au vert, le pipeline a appelé le Deploy Hook ;
Render et Vercel ont reconstruit et mis en ligne la nouvelle version automatiquement.

GitHub Actions du frontend



Sur le tableau de bord de Render, on voit que les déploiements sont bien déclenchés par le pipeline : la colonne « Trigger » indique « Deploy Hook ».




Ce que m'apporte cette démarche

Fiabilité : le code est vérifié automatiquement à chaque modification, de la même façon à chaque fois.

Rapidité : il suffit d'un push pour que la nouvelle version soit en ligne en environ 1 à 2 minutes.

Sécurité : les secrets ne sont jamais dans le code, et une version qui ne passe pas les tests n'est jamais déployée.

Traçabilité : chaque déploiement est lié à un commit précis, visible dans GitHub, Render et Vercel.

**2. Précisez les moyens utilisés :**

**Git, GitHub, GitHub Actions, Docker, ESLint, Jest, Vitest, Render, Vercel**

**3. Avec qui avez-vous travaillé ?**

**J'ai travaillé avec mon tuteur et l'IA Claude**

**4. Contexte**

**Nom de l'entreprise, organisme ou association** ▸ ***Global Digital University***

**Chantier, atelier, service** ▸ **Contribuer à la mise en production dans une démarche DevOps**

**Période d'exercice** ▸ **Du 26/05/2026 au 06/09/2026**

**5. Informations complémentaires** *(facultatif)*

Cliquez ici pour taper du texte.

# Titres, diplômes, CQP, attestations de formation

*(facultatif)*

| Intitulé | Autorité ou organisme | Date |
|---|---|---|
| **Licence pro Mécatronique** | **IUT Troyes** | **2018** |
| Cliquez ici. | Cliquez ici pour taper du texte. | Cliquez ici pour sélectionner une date. |
| Cliquez ici. | Cliquez ici pour taper du texte. | Cliquez ici pour sélectionner une date. |
| Cliquez ici. | Cliquez ici pour taper du texte. | Cliquez ici pour sélectionner une date. |
| Cliquez ici. | Cliquez ici pour taper du texte. | Cliquez ici pour sélectionner une date. |
| Cliquez ici. | Cliquez ici pour taper du texte. | Cliquez ici pour sélectionner une date. |
| Cliquez ici. | Cliquez ici pour taper du texte. | Cliquez ici pour sélectionner une date. |
| Cliquez ici. | Cliquez ici pour taper du texte. | Cliquez ici pour sélectionner une date. |
| Cliquez ici. | Cliquez ici pour taper du texte. | Cliquez ici pour sélectionner une date. |
| Cliquez ici. | Cliquez ici pour taper du texte. | Cliquez ici pour sélectionner une date. |

# Déclaration sur l'honneur

Je soussigné(e) [prénom et nom] *Elia SOINARD*,

déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je suis l'auteur(e) des réalisations jointes.

Fait à *PHALSBOURG* le 18/10/2026

pour faire valoir ce que de droit.

Signature :



# ANNEXES

*(Si le RC le prévoit)*